

# SEMI-SUPERVISED TUNING FOR CLUSTERING



Tonet Lorenzo - Progetto finale



[Repository GitHub](#)

# PROBLEMA PROPOSTO:

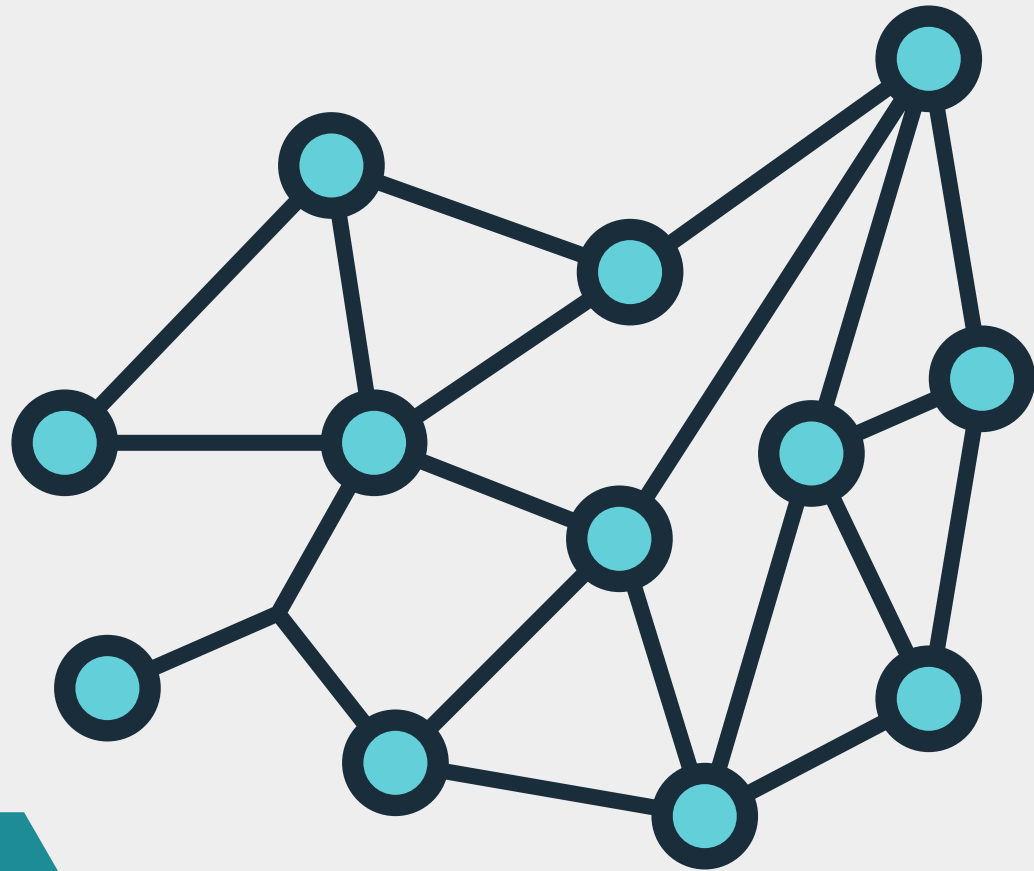
“Come è possibile sfruttare le informazioni (anche se poche) sui dati per una task di tipo unsupervised?”



# PROGRAMMA

- Introduzione Teorica
- Architettura modelli
- Approccio proposto
- Risultati

# SPECTRAL CLUSTERING



- Costruzione grafo dei dati in cui i pesi degli archi sono una misura della similarità
- Costruzione matrice Laplaciana del grafo

$$L = D - W$$

- Calcolo dei primi k autovalori di L ordinati in modo crescente e i relativi autovettori
- Questi autovettori formano un embedding dei dati su cui fare clustering

$$W_{ij} = \text{similarity}(x_i, x_j)$$
$$D_{ii} = \text{degree}(x_i)$$

# SIAMESE NETWORKS

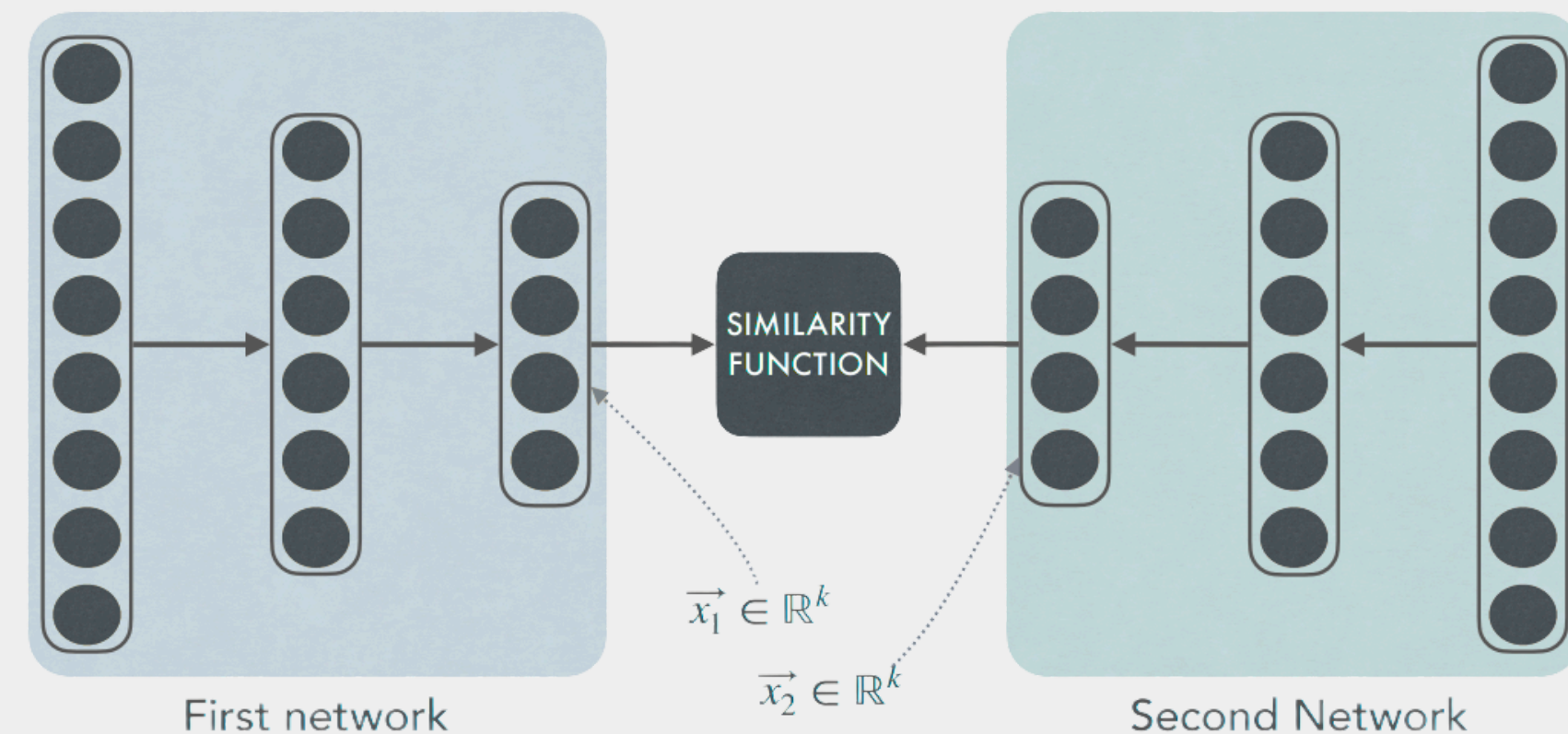
## CARATTERISTICHE:

- Pesi condivisi (di fatto è una rete sola)
- 2 o più input diversi
- La loss sarà qualcosa che misura una relazione tra l'output della rete per gli input (come per esempio una distanza)

## CONTRASTIVE LOSS:

$$Loss = (1 - y)D^2 + y \times \max(0, m - D)^2$$

- $y = 0$  per coppia simile, 1 per coppia diversa
- $m$  = margine threshold per la similarità
- $D$  = distanza tra embeddings (euclidea solitamente)





# APPROCCIO PROPOSTO

## Training

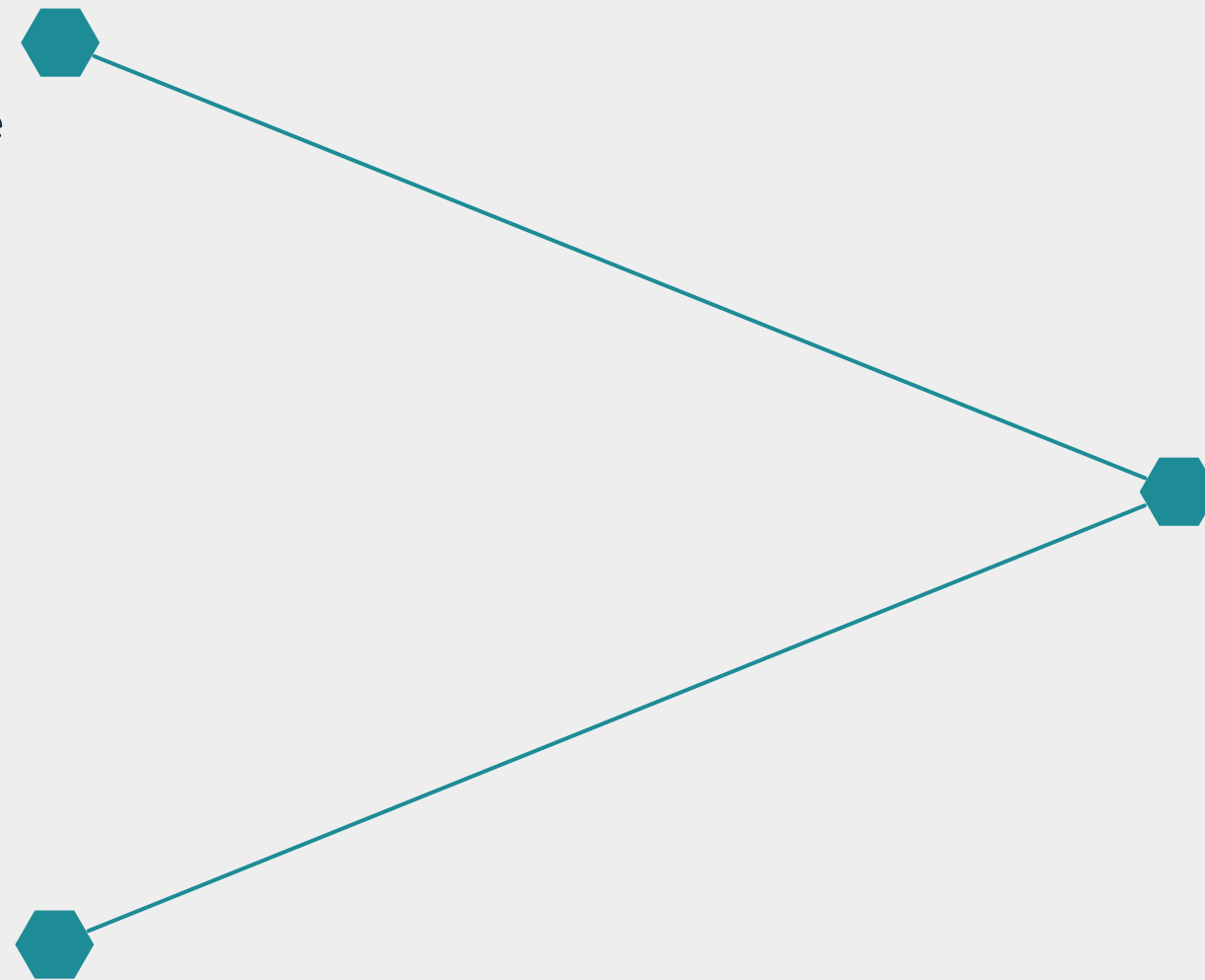
La rete neurale viene allenata utilizzando la parte labeled

## Embeddings

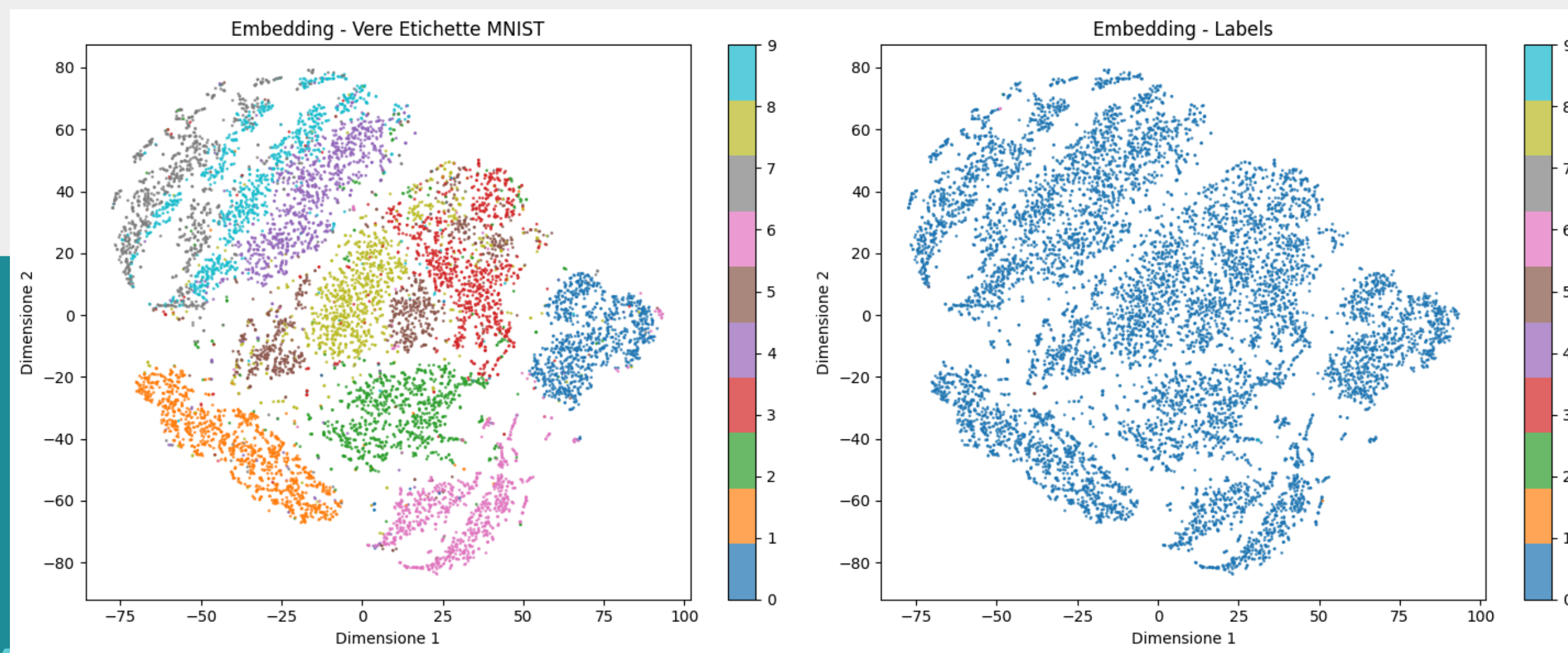
La rete imparata viene utilizzata per creare gli embedding dei dati non labeled in uno spazio n-dimensionale (n sarà un parametro)

## Spectral clustering

Viene sfruttato lo spazio imparato per calcolare la similarità e costruire il grafo



# CLUSTERING ON BASELINE (NO-EMBEDDINGS)



PCA(64)

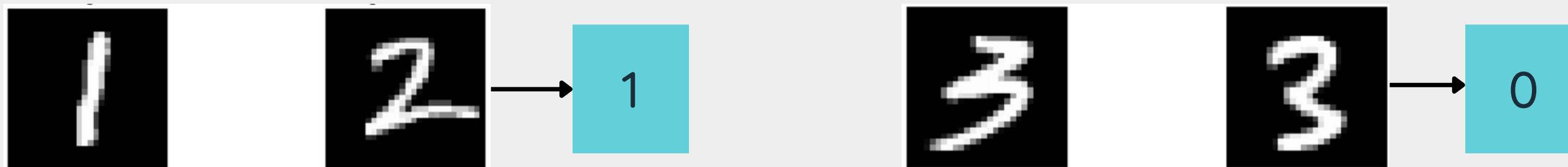
Spectral clustering

T-SNE (perp = 30)  
(for visualization)

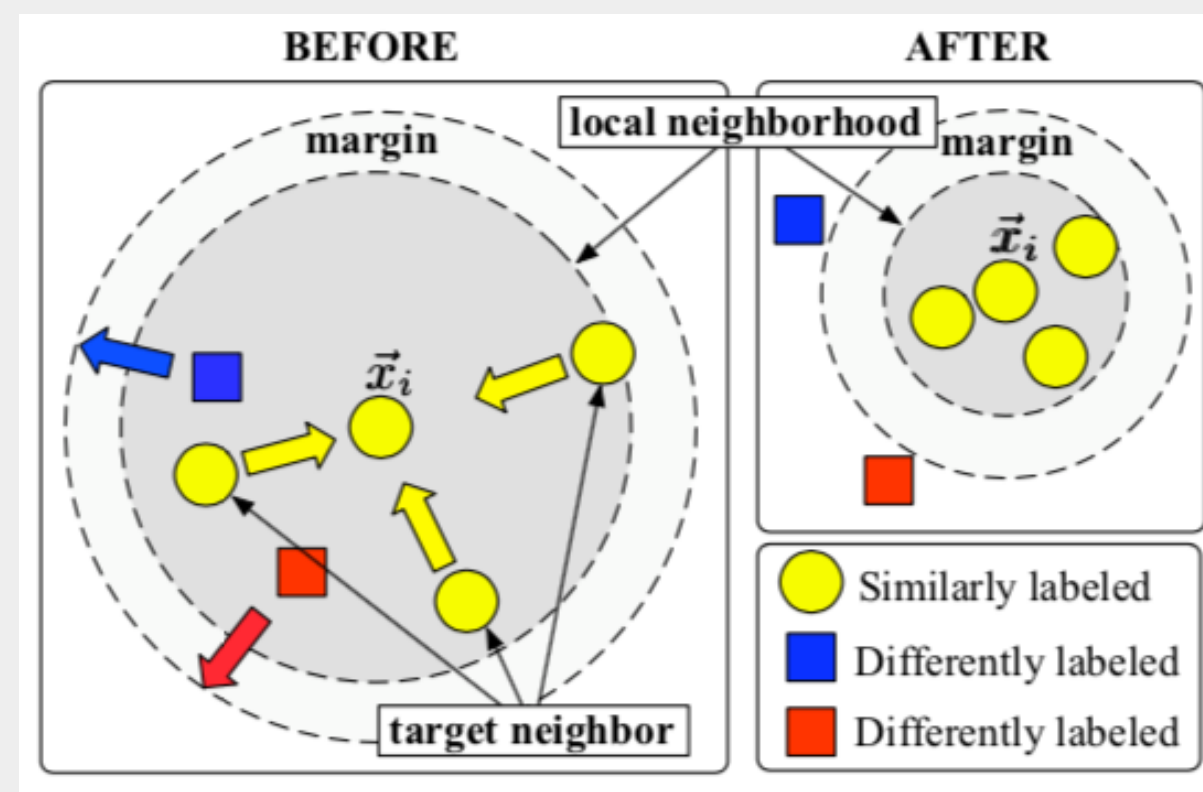


# CONTRASTIVE WEIGHTED LOSS

Forma del dataset:



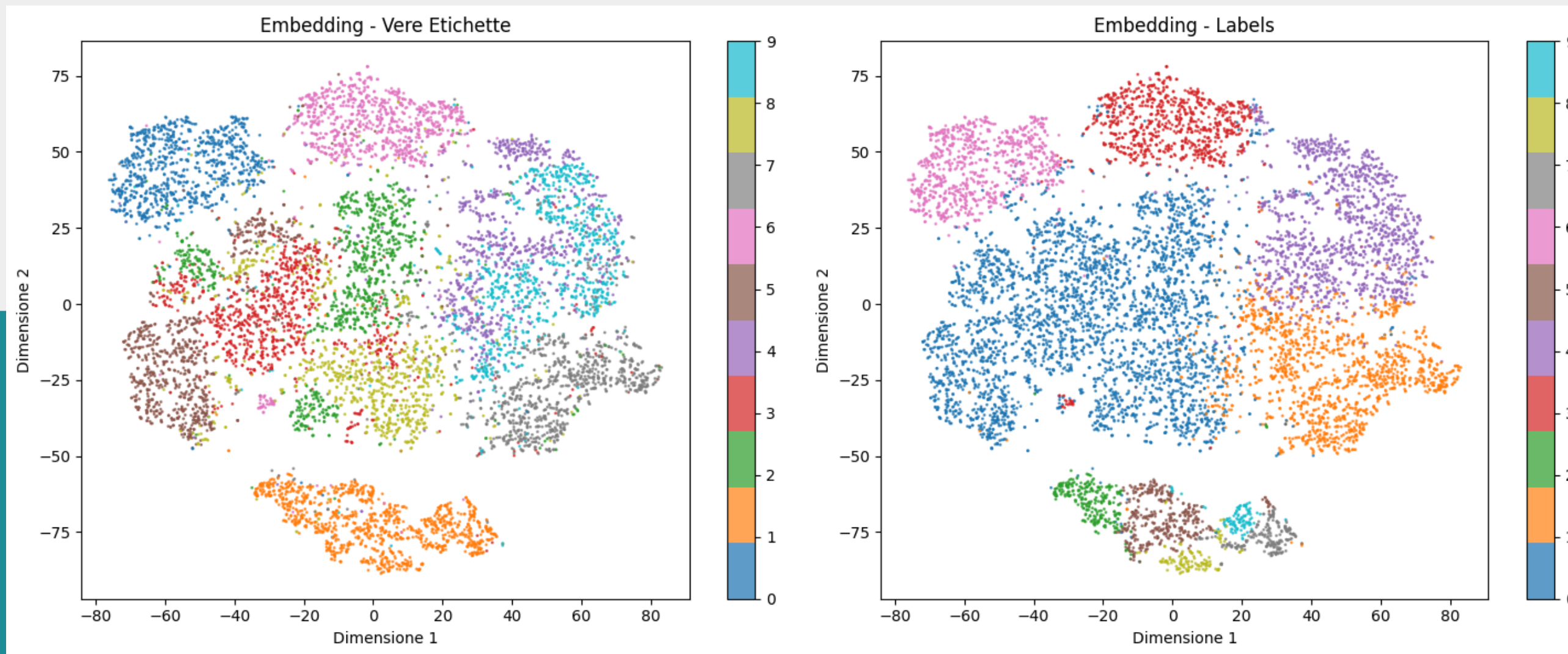
$$Loss = (1 - y)D^2 + y \times \max(0, m - D)^2 \times w$$



Peso utilizzato per penalizzare di più le coppie diverse



# SIAMESE NETWORK → CLUSTERING



Data = MNIST

Loss: Weighted Contrastive (margin = 2, w = 3)

Partition Labeled: 100%

Batch size = 128

Dummy accuracy = 56,4%

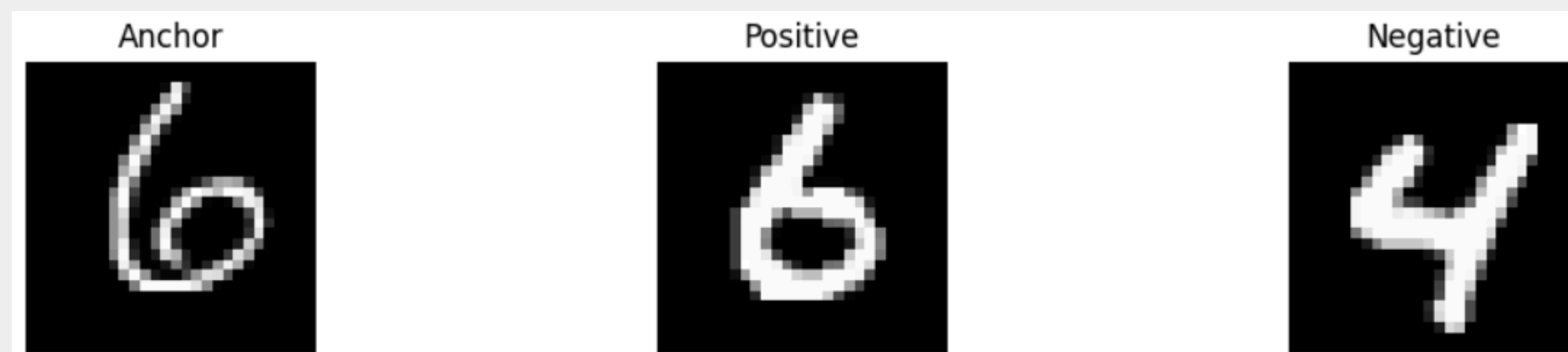
Embedding dim: 64

Learning rate: 1e-5

-----  
TSNE(perplexity = 30)

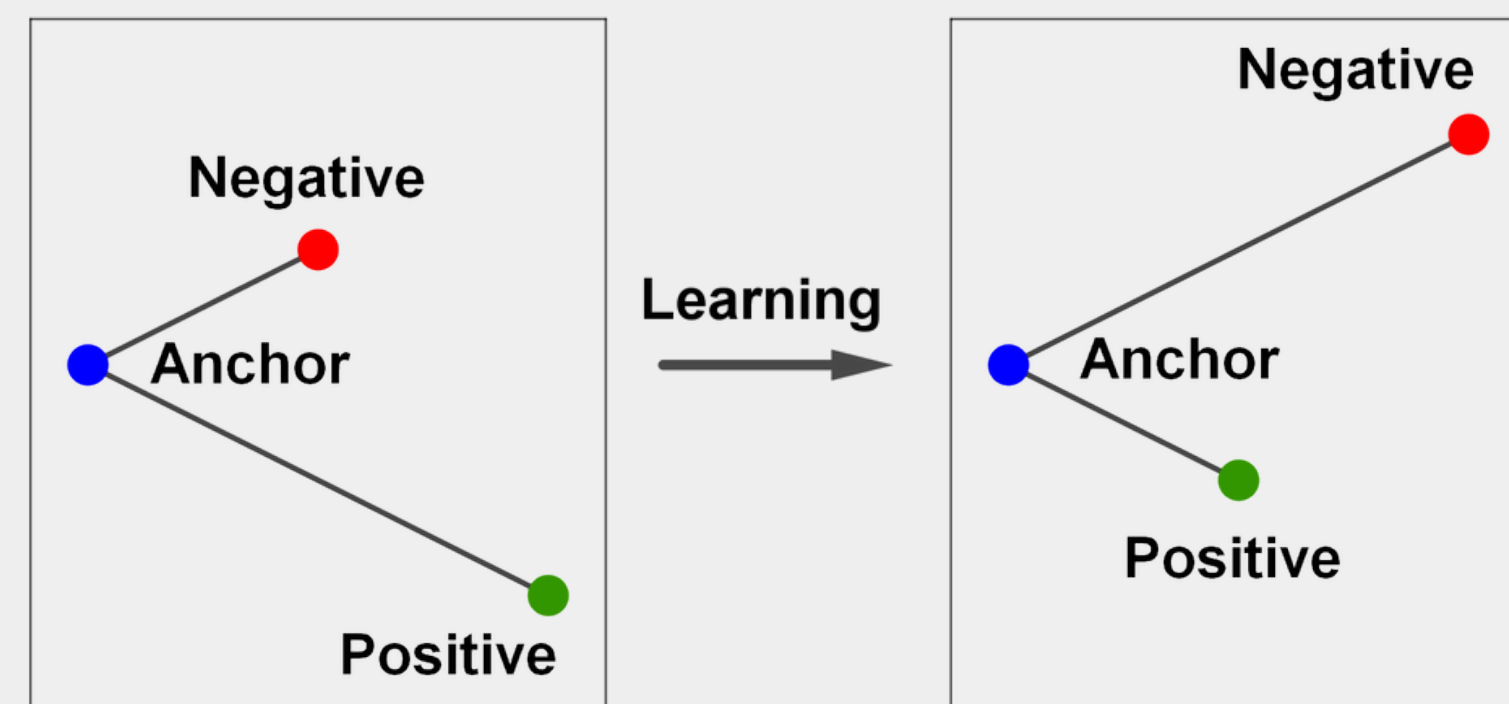
# TRIPLER MARGIN LOSS

Forma del dataset:

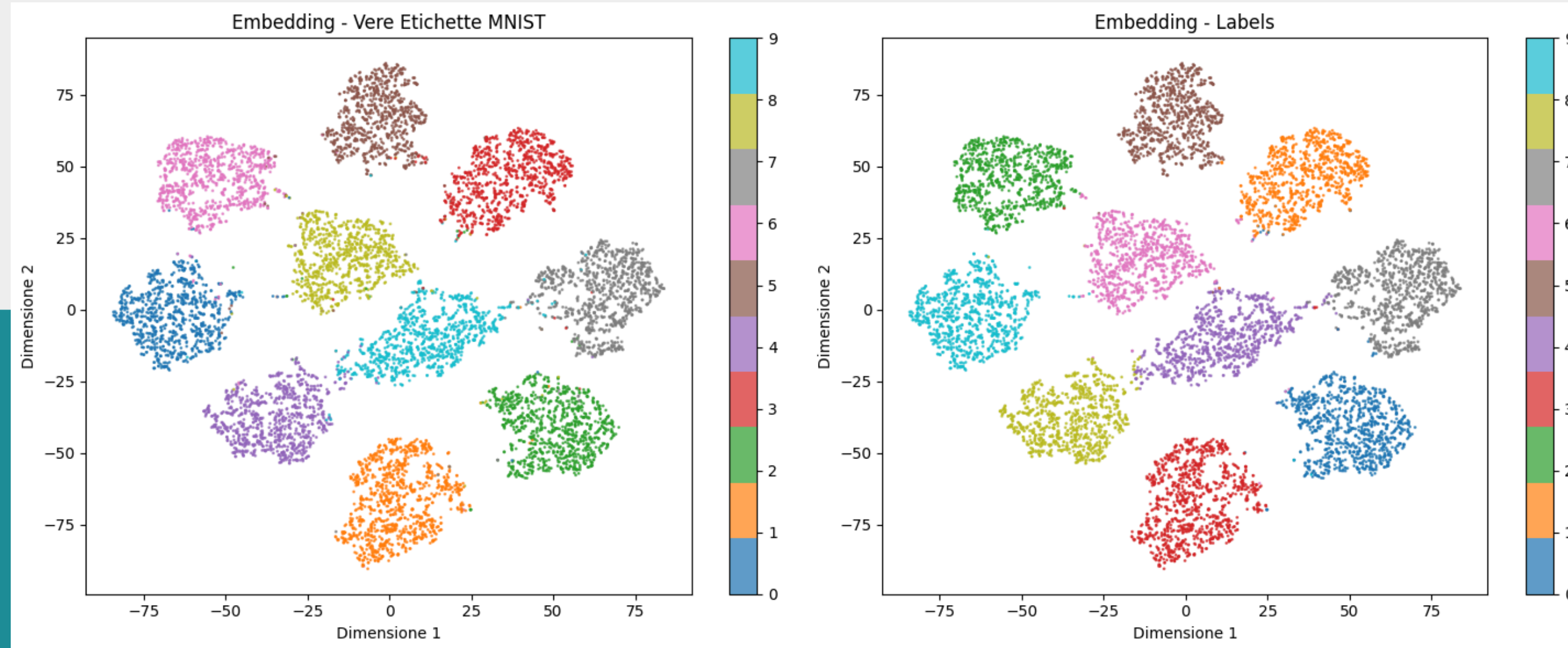


$$Loss = \max [d(a, p) - d(a, n) + \text{margin}, 0]$$

$$d(x, y) = \|x - y\|_p$$



# SIAMESE NET. (TRIPLER LOSS) → CLUSTERING

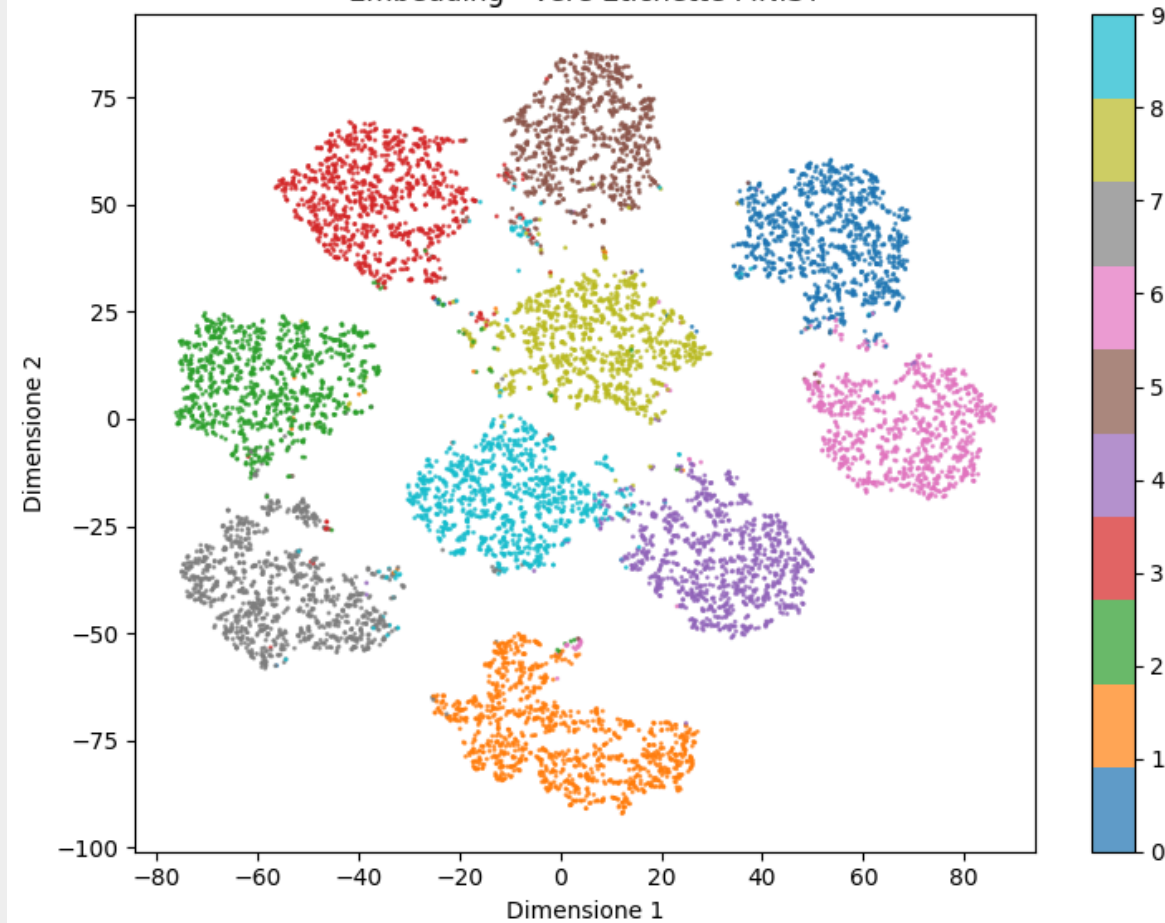


Data = MNIST  
Loss: Triplet Margin (margin = 2)  
Partition Labeled: 20%  
Batch size = 128  
Dummy accuracy = 98,8%  
Embedding dim: 64  
Learning rate: 1e-4

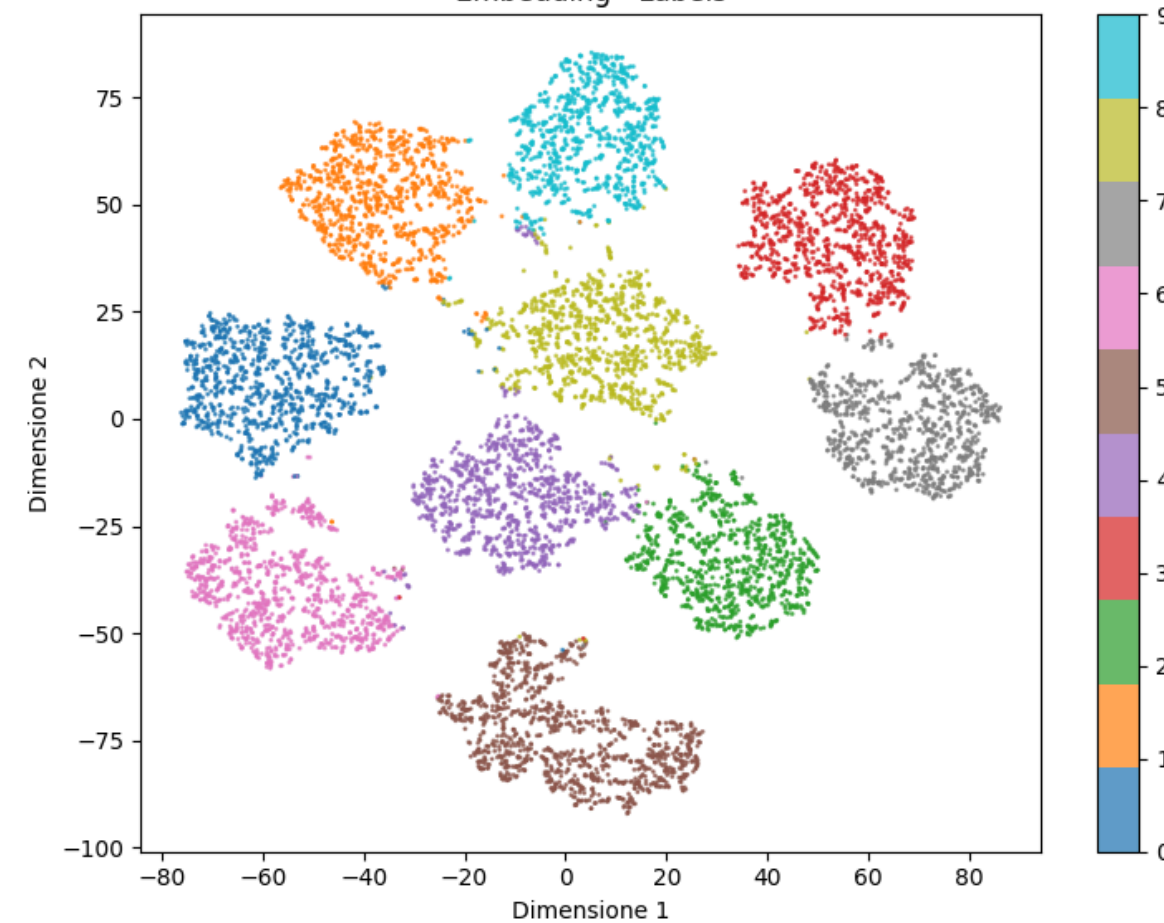
-----  
TSNE(perplexity = 30)



Embedding - Vere Etichette MNIST



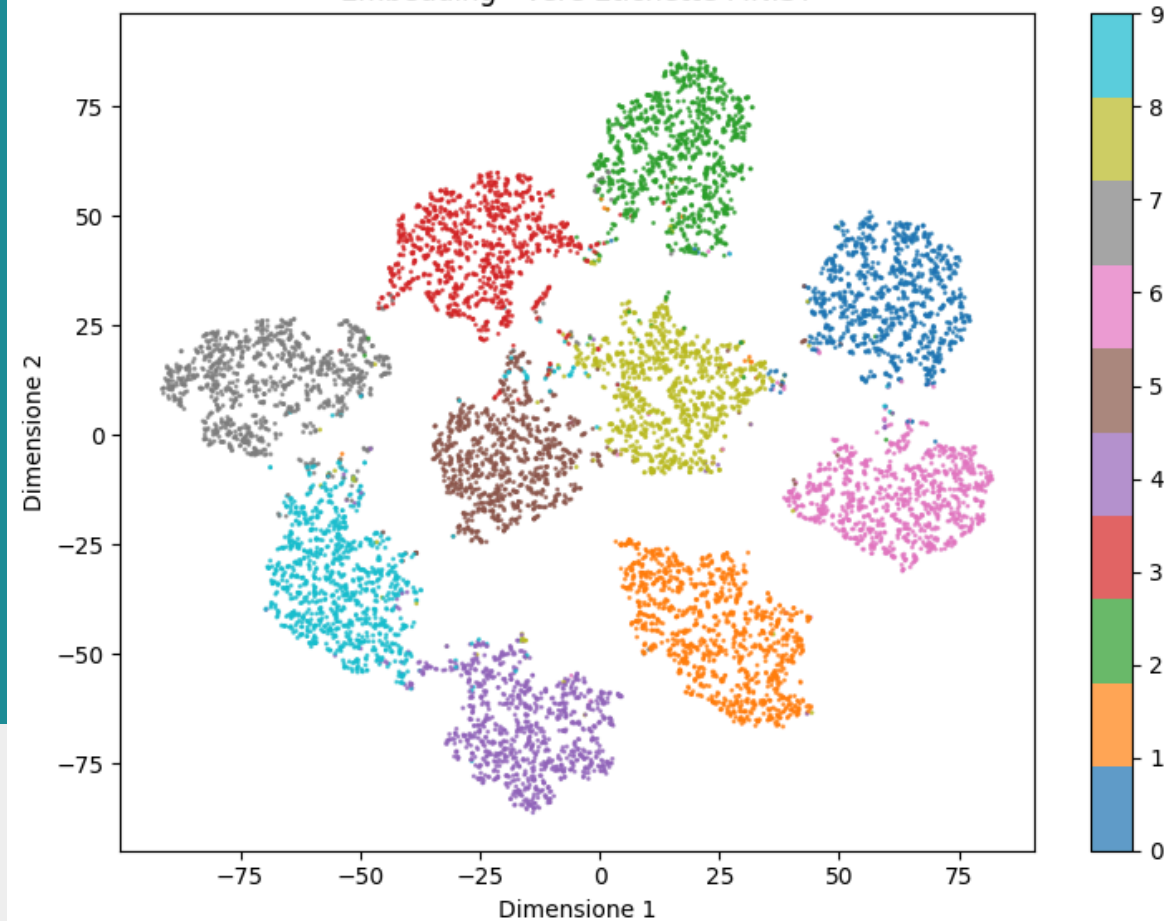
Embedding - Labels



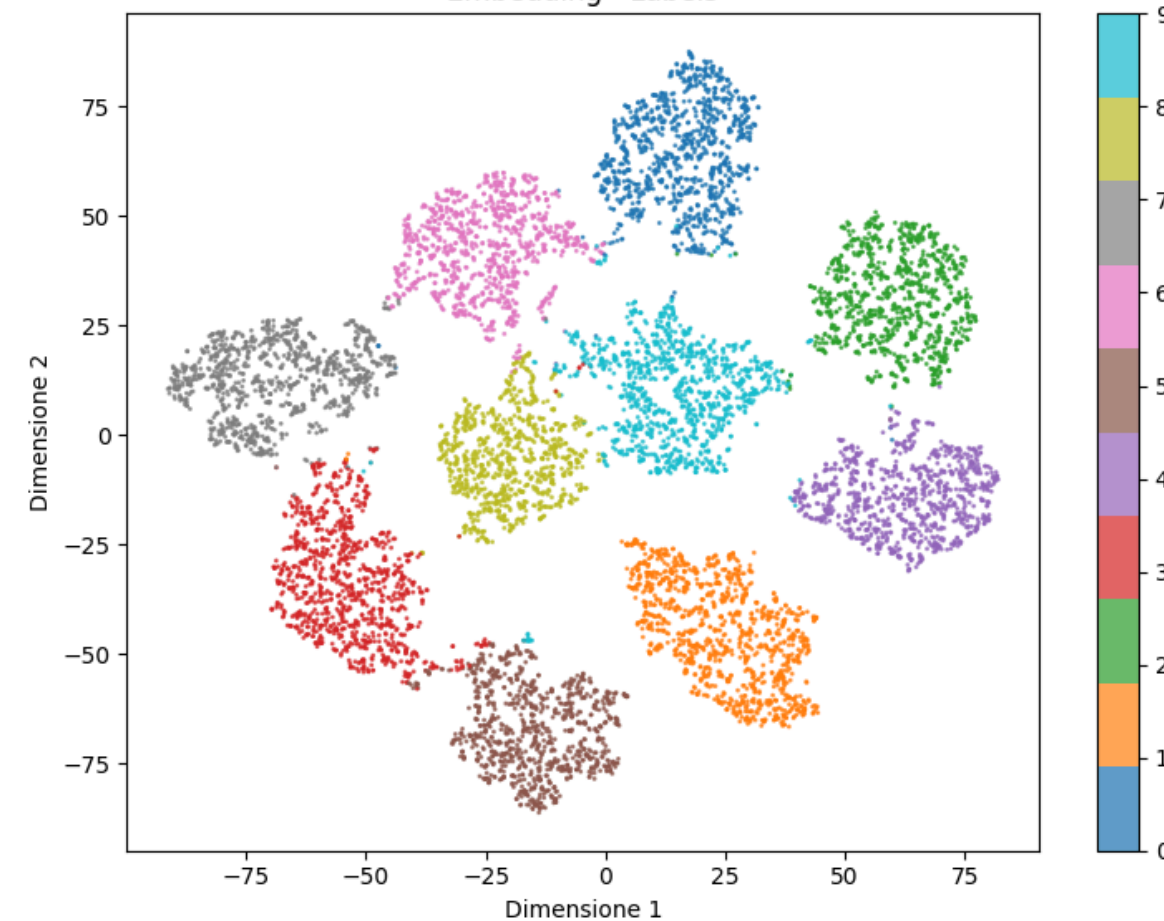
Data = MNIST  
 Loss: Triplet Margin (margin = 2)  
 Partition Labeled: 10%  
 Batch size = 128  
 Dummy accuracy = 99,1%  
 Embedding dim: 64  
 Learning rate: 1e-4

-----  
 TSNE(perplexity = 30)

Embedding - Vere Etichette MNIST



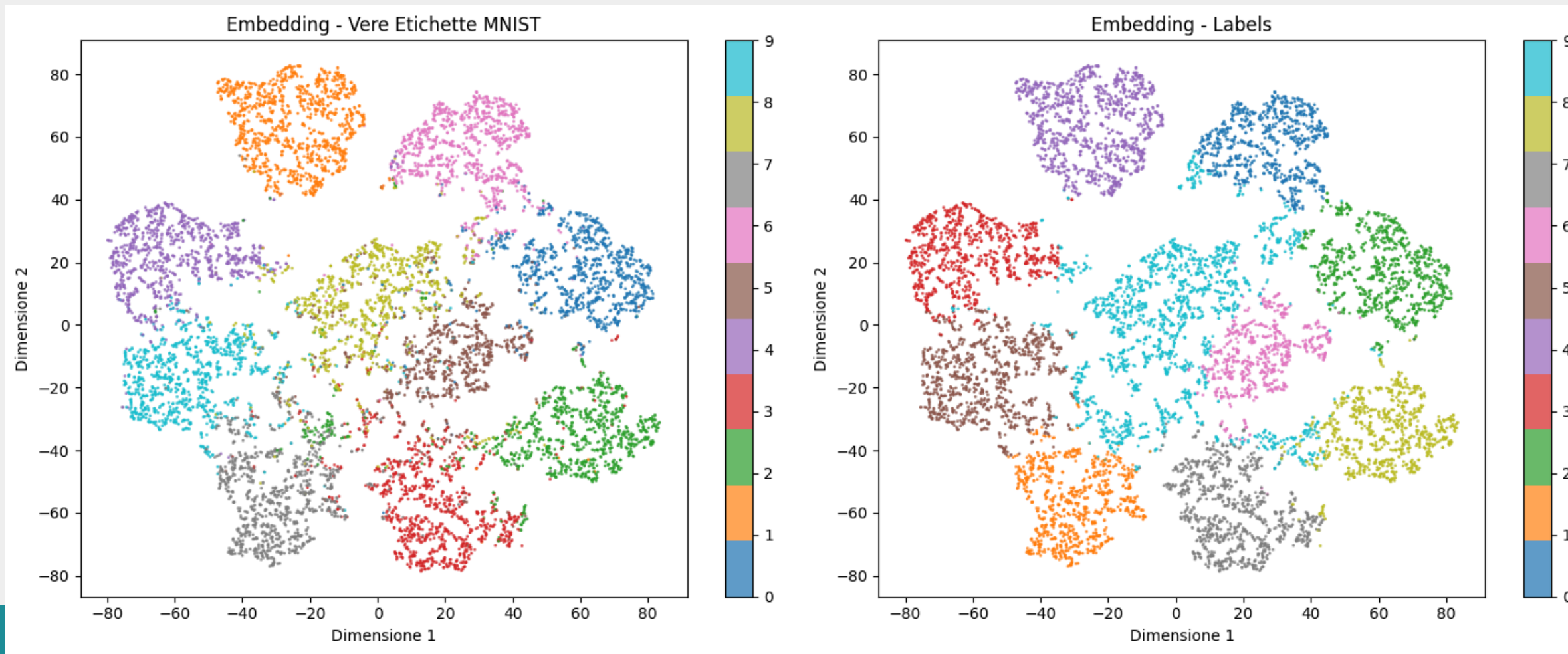
Embedding - Labels



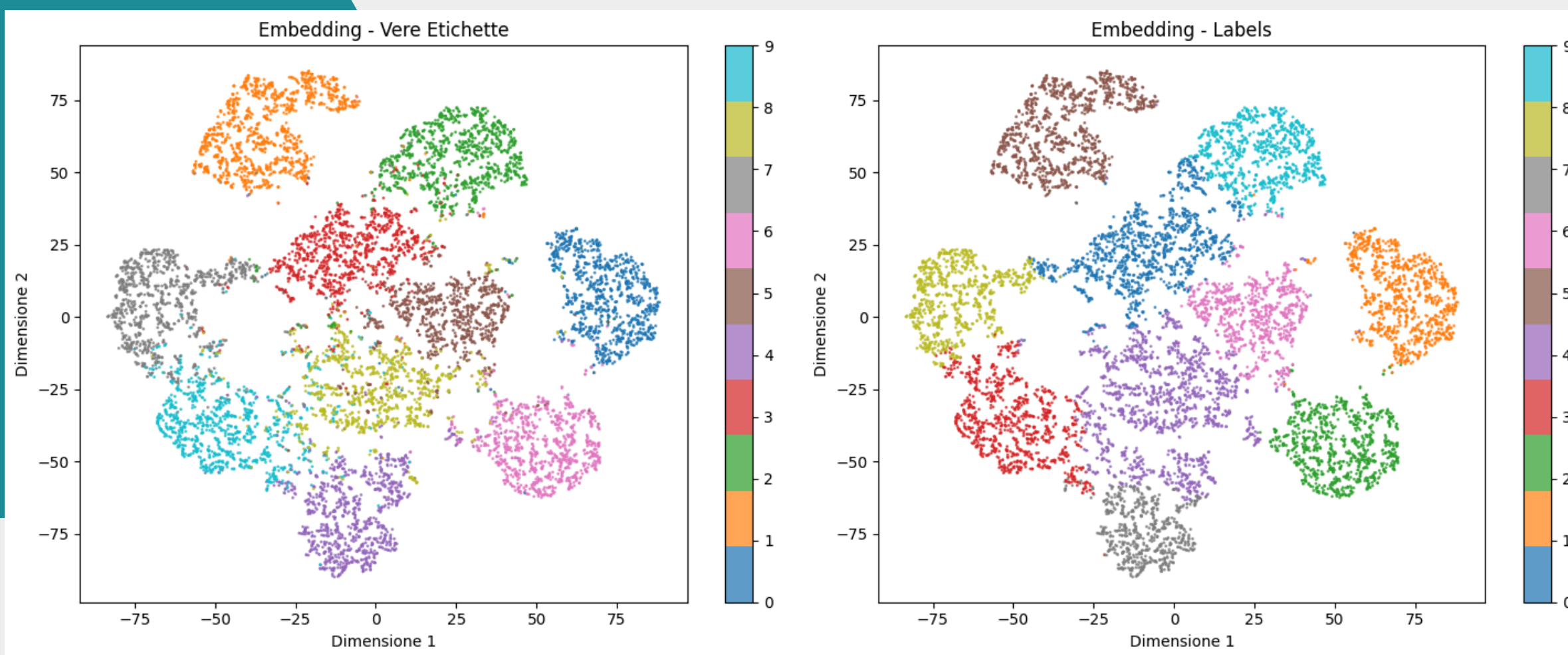
Data = MNIST  
 Loss: Triplet Margin (margin = 2)  
 Partition Labeled: 5%  
 Batch size = 32  
 Dummy accuracy = 97.7%  
 Embedding dim: 64  
 Learning rate: 1e-4

-----  
 TSNE(perplexity = 30)





Data = MNIST  
Loss: Triplet Margin (margin = 2)  
Partition Labeled: 1%  
Batch size = 64  
Dummy accuracy = 92%  
Embedding dim: 64  
Learning rate: 1e-4  
 -----  
 TSNE(perplexity = 30)



Data = MNIST  
Loss: Triplet Margin (margin = 2)  
Partition Labeled: 1% ---> (triplets mined = 5000)  
Batch size = 128  
Dummy accuracy = 95,6%  
Embedding dim: 64  
Learning rate: 1e-4  
 -----  
 TSNE(perplexity = 30)

# RISULTATI

## MNIST

| Metodo                 | ARI   | NMI   | R@1    | R@2    | R@4    | R@8    |
|------------------------|-------|-------|--------|--------|--------|--------|
| Couple – 100%          | 0.358 | 0.600 | 0.8887 | 0.9416 | 0.9710 | 0.9844 |
| Triplet – 20%          | 0.955 | 0.945 | 0.9766 | 0.9867 | 0.9915 | 0.9945 |
| Triplet – 10%          | 0.932 | 0.922 | 0.9655 | 0.9814 | 0.9893 | 0.9931 |
| Triplet – 5%           | 0.920 | 0.912 | 0.9615 | 0.9769 | 0.9852 | 0.9912 |
| Triplet – 1%           | 0.709 | 0.776 | 0.8836 | 0.9308 | 0.9574 | 0.9717 |
| Triplet – 1% – offline | 0.744 | 0.794 | 0.9267 | 0.9561 | 0.9735 | 0.983  |

## FASHION MNIST

| Metodo                 | ARI   | NMI   | R@1    | R@2    | R@4    | R@8    |
|------------------------|-------|-------|--------|--------|--------|--------|
| Triplet – 20%          | 0.564 | 0.693 | 0.7690 | 0.8573 | 0.9155 | 0.9512 |
| Triplet – 10%          | 0.529 | 0.669 | 0.7604 | 0.8541 | 0.9160 | 0.9546 |
| Triplet – 1%           | 0.348 | 0.541 | 0.6429 | 0.7656 | 0.8641 | 0.9308 |
| Triplet – 1% – offline | 0.363 | 0.556 | 0.7167 | 0.8139 | 0.8857 | 0.9346 |

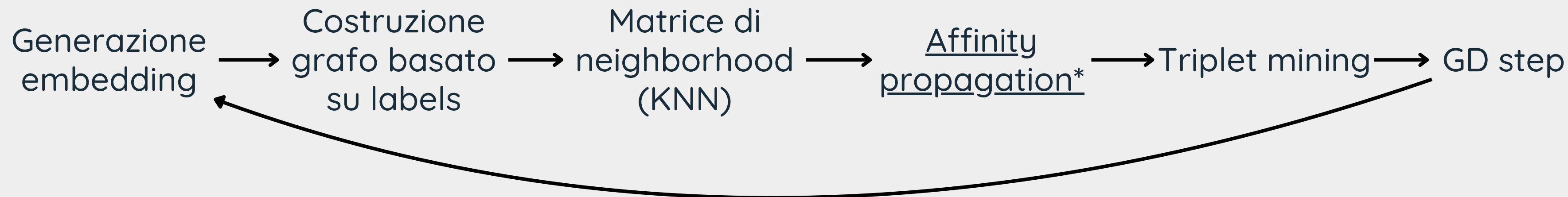


# MIGLIORAMENTI PROPOSTI: MIGLIORARE IL “TRIPLER MINING”

## Problema

Le triplete potrebbero essere poco informative oppure “troppo facili” quindi il modello potrebbe smettere di imparare

## Training loop proposto dal paper:



### Semi-Supervised Metric Learning: A Deep Resurrection

Ujjal Kr Dutta,<sup>1,2</sup> Mehrtash Harandi,<sup>3</sup> C Chandra Sekhar<sup>2</sup>

<sup>1</sup> Data Sciences, Myntra, India

<sup>2</sup>Dept. of Computer Science and Eng., Indian Institute of Technology Madras, India

<sup>3</sup>Dept. of Electrical and Computer Systems Eng., Monash University, Australia

ukd@cse.iitm.ac.in, ujjal.dutta@myntra.com, mehrtash.harandi@monash.edu, chandra@cse.iitm.ac.in

#### Abstract

Distance Metric Learning (DML) seeks to learn a discriminative embedding where similar examples are closer, and dissimilar examples are apart. In this paper, we address the problem of Semi-Supervised DML (SSDML) that tries to learn a metric using a few labeled examples, and abundantly available unlabeled examples. SSDML is important because

in nature. In order to provide constraints (pairs/ triplets/ tuples) for metric learning, they require manual annotations (*class labels*). For large scale datasets, it is infeasible to obtain class labels for all the examples. In practical scenarios where it is possible to have a few examples annotated, with an abundance of additional unlabeled data, *semi-supervised learning* (Oliver et al. 2018; Zhu 2005; Chapelle, Scholkopf, and Zien 2009; Stretcu et al. 2019; Miyato et al. 2018; Iscen

### Affinity propagation

$$W^* = (1 - \gamma)(\mathbf{I}_{N_L + N_p} - \gamma Q)^{-1} W^0$$

$Q$  = neighborhood matrix

$W^0$  = affinity matrix (labels)

$\gamma$  = weight (tra 0 e 1)





# FINE

## Bibliografia:

- A Tutorial on Spectral Clustering - Ulrike von Luxburg  
<https://arxiv.org/abs/0711.0189>
- Semi-Supervised Metric Learning: A Deep Resurrection - Ujjal Kr Dutta, Mehrtash Harandi, Chellu Chandra Sekhar  
<https://arxiv.org/abs/2105.05061>

## Codice:

<https://github.com/LorenzoTonet/Intro-to-Machine-Learning---Exam-Project>